VOID CTF — Forensics Writeups

Author: Akshat (@akshat-xcode)

Team: Code Crew

Category: Forensics

Points: 1400 / 1400 (All 5 Solved)

Overview

Challenge	Points	Topic	Flag
I Heard something	100	Audio Spectrogram Analysis	`VOID{y0u_c4n7_h34r_m3}`
Left Behind	200	USB HID Keystroke Sniffing	`VOID{h1d_k3ystr0k3s}`
A Sealed Record	300	PNG IHDR Height & CRC32 Repair	`VOID{crc_r3v34ls_th3_tru3_h31ght}`
Volatile Mira	300	LiME Memory Forensics & XOR Decryption	`VOID{f0r3n51c5_15_4n_4r7}`
3000 Files Later	500	ZIP Epoch & SHA-256 Signature Verification	`VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}`

---

1. I Heard something (100 pts)

Description

> *"Mira left a voice note on her desktop called 'whisper.' It's just a tune. Or is it? Some things are only meant for machines to hear."*

Solution

We were given an audio file `whisper.wav`. Listening to the audio, it just plays a simple acoustic tune. The hint says: "*Some things are only meant for machines to hear.*"

Since adult human hearing tops out around 15–16 kHz, I inspected the high-frequency range using a spectrogram. Using Python with `scipy` and `matplotlib`:

```
import scipy.io.wavfile as wavfile
import matplotlib.pyplot as plt

sr, data = wavfile.read("whisper.wav")

plt.figure(figsize=(12, 5))
plt.specgram(data, NFFT=2048, Fs=sr, noverlap=1024, cmap='viridis')
plt.ylim(15000, 20000) # Zoom into ultrasonic band
plt.title("High Frequency Spectrogram")
plt.savefig("spectrogram.png")
```

In the resulting spectrogram between 2s and 10s at ~16.3–19.3 kHz, the flag was drawn directly as visual text:

Flag: `VOID{y0u_c4n7_h34r_m3}`

---

2. Left Behind (200 pts)

Description

> *"Arlo Vance runs the server room at Halvorsen Logistics and trusts nothing that touches a network. His workstation is air-gapped, and his passwords live only in his head. During last night's audit, someone left a small device plugged in behind his desk. It sat there quietly until Arlo noticed it this morning and yanked it out. This file is all it left behind. Arlo insists there was nothing on it worth worrying about. Prove him wrong."*

Solution

We received `left_behind.pcapng`. Opening it in Wireshark showed it contains raw USB traffic captured via Linux `usbmon` (Linktype 220), not network packets.

Looking at the USB descriptors:

- Device 4 is a Logitech USB Keyboard (`VID 0x046d, PID 0xc31c`) sending 8-byte interrupt reports on endpoint `0x81`.
- The device left behind was an inline USB keylogger.

In the standard USB HID keyboard boot protocol:

- Byte 0 is the modifier bitmask (Shift, Ctrl, Alt).
- Bytes 2–7 contain key usage IDs.

I wrote a python script to parse the USB packets and reconstruct Arlo's typing:

- He started typing: `VOID{h1d_`
- Then made a typo: `wrong`
- Pressed **Backspace (`0x2a`) 5 times** to delete `wrong`
- Continued typing: `k3ystr0k3s}`

Flag: `VOID{h1d_k3ystr0k3s}`

---

3. A Sealed Record (300 pts)

Description

> *"An evidence image came in from intake, but something's off — a few tools refuse to open it at all. The analyst who logged it left one note: 'the tag was rewritten after sealing. the seal still remembers the original. trust the seal, not the tag.' Recover the record."*

Solution

We were given `Artifact-BC34.rar`, which extracts to `Artifact-BC34.png`. Several image viewers and forensic tools gave errors when opening it.

The analyst note explains why:

> *"the tag was rewritten after sealing. the seal still remembers the original. trust the seal, not the tag."*

In PNG files, the `IHDR` chunk contains width and height (the "tag"), followed by a CRC32 checksum (the "seal"). The height had been modified without recomputing the CRC:

- Width: `2464` px
- Tampered Height: `1607` px
- Stored `IHDR` CRC: `0x14C4F49F`

Since "the seal still remembers the original", I wrote a quick script to brute-force the original height by matching the CRC:

```
import struct, zlib

width = 2464
stored_crc = 0x14C4F49F

for h in range(1, 10000):
    ihdr = b'IHDR' + struct.pack('>IIBBBBB', width, h, 8, 2, 0, 0, 0)
    if zlib.crc32(ihdr) == stored_crc:
        print(f"[+] Found real height: {h}") # 1728
        break
```

The original height was **1728 px** (the image had been cropped by 121 px). I patched bytes `0x14–0x17` in the PNG from `00 00 06 47` to `00 00 06 C0`. Opening the fixed image reveals the bottom 121 rows with a terminal banner containing the flag:

Flag: `VOID{crc_r3v34ls_th3_tru3_h31ght}`

---

4. Volatile Mira (300 pts)

Description

> *"Mira never writes secrets to disk. 'Disk is evidence,' she wrote in her diary. 'RAM forgets.' Her notes daemon keeps everything encrypted in memory, and she swears the key never sits next to the vault. When her laptop went dark, our responder didn't pull the plug. She grabbed a full image of the machine's RAM first. Mira was wrong about one thing: RAM forgets only when the power goes. Find what she was hiding."*

Solution

We were given `mem.rar`, which extracts to a 2 GB LiME Linux memory dump (`mem.lime`).

1. **Finding the Daemon:**

Searching memory for Python scripts and keywords like `vault` recovered the script `/opt/mira/heartbeat.py`:

```python
SEALED = bytes.fromhex("632c7f761e530146035b020402566804003c025c3a0143034d")
vault = bytearray(b"MIRA-VAULT::" + SEALED + b"::END")
```

2. **Finding the Key:**

The hint says "*the key never sits next to the vault*", meaning the key wasn't hardcoded in the file. Checking process command-line arguments and bash history in memory revealed how the script was launched:

```bash
./launch.sh 5c62e5140575ac75
shred -u launch.sh
python3 /opt/mira/heartbeat.py --session-token 5c62e5140575ac75 --interval 60
```

The key is the session token: `5c62e5140575ac75`.

3. **Decryption:**

Decrypting the 25-byte `SEALED` ciphertext with repeating-key XOR:

```python
SEALED = bytes.fromhex("632c7f761e530146035b020402566804003c025c3a0143034d")
```

key = b"5c62e5140575ac75"

flag = bytes([c ^ key[i % len(key)] for i, c in enumerate(SEALED)])

print(flag.decode())

```

**Flag:** `VOID{f0r3n51c5_15_4n_4r7}`

---

## 5. 3000 Files Later (500 pts)

### Description

> *"Someone poured their whole life into a zip file. Thousands of flags, every one of them convincing. They swear only one is real, and that they left a way to prove it. Note - Before you start grepping: they saw you coming."*

### Solution

The zip archive contains 3,369 files. Every single file has a Base64 string that decodes to a valid-looking flag matching `VOID{...}`, so grepping is useless.

I checked the ZIP archive comment:

> `"Ask the one that marks the beginning of an era how to sign. Trust the youngest who bears the mark."`

1. **"Beginning of an era":**

In ZIP/FAT filesystem format, the earliest possible timestamp (the epoch) is `1980-01-01 00:00:00`.

Looking at the timestamps of all entries, exactly one file has this timestamp: `ae/4a/d9dc427dacb85135.txt`.

Decoding its content gives:

`VOID{my_t41l_th3n_y0ur_b0dy_sha256_f1rst_8_1s_y0ur_t41l_1e7ea419}`

This reveals the signature rule:

`SHA256("1e7ea419" + body)[:8] == tail`

2. **Finding Valid Candidates:**

I wrote a script to test all 3,369 files against this SHA-256 rule. Exactly **8 files** passed the signature check.

3. **"Trust the youngest":**

Sorting the 8 valid candidates by modification time, the newest (youngest) file is:

- **Timestamp:** `2015-02-04 06:30:32`

- **File:** `b1/ab/54b611bcdb191433.txt`

- **Content:** `VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}`

**Flag:** `VOID{y0u_ch3ck3d_3000_f1l3s_f0r_th1s_021959f6}`
```